

Structuring Machine Learning Projects

A Strategy Reference — Approaches & Decision Frameworks

This reference distills the core strategy concepts for diagnosing and improving machine learning systems — drawn from a worked set of practice questions and accompanying selected course notes from Stanford’s Deep Learning modules. It is organized around the practical decisions a project lead actually faces: what to measure, how to split data, how to read bias and variance, and which learning paradigm fits a given problem.

Contents

1. Orthogonalization: The Organizing Principle	2
2. Evaluation Metrics	2
2.1 Use a single-number evaluation metric	2
2.2 Optimizing vs. satisficing metrics	2
2.3 When to change the metric or the dev/test set	3
3. Setting Up Train / Dev / Test Sets	4
3.1 Dev and test must come from the same distribution	4
3.2 Size dev/test in absolute terms, not by ratio	4
3.3 Put the right data in the right place	4
4. Human-Level Performance, Bias & Variance	4
4.1 Why human-level performance matters	4
4.2 The bias / variance decomposition	4
4.3 Bayes error and surpassing humans	5
5. Bias & Variance with Mismatched Distributions	6
6. Workflow & Error Analysis	6
6.1 Build your first system quickly, then iterate	6
6.2 Error analysis	6
6.3 Cleaning up incorrectly labeled data	6
7. Choosing a Learning Paradigm	8
7.1 Transfer learning	8
7.2 Multi-task learning	8
7.3 End-to-end deep learning	8
8. One-Page Diagnostic Cheat Sheet	9

1. Orthogonalization: The Organizing Principle

Effective ML practitioners are clear-eyed about **which knob to turn to achieve which effect**. Orthogonalization means each control adjusts one thing, the way a car separates steering from acceleration — so you can diagnose and fix one problem without disturbing the others. The whole workflow below is an application of this idea: aim at the right target first, then optimize toward it.

The chain of conditions a supervised system should satisfy, each with its own dedicated “knob”:

- **Fit the training set well** (near human-level). Knobs: bigger network, better optimizer, train longer.
- **Fit the dev set well**. Knobs: regularization, more training data.
- **Fit the test set well**. Knob: a bigger dev set (a small gap here means you over-tuned to dev).
- **Perform well in the real world**. Knobs: change the dev/test set or change the cost metric.

Key idea Before improving a system, confirm you are aiming at the right target. A misaligned metric or dev set is an aiming problem, not an optimization problem — and no amount of tuning fixes a target pointed in the wrong direction.

2. Evaluation Metrics

2.1 Use a single-number evaluation metric

A single real-valued metric lets you compare models instantly and iterate faster. When two metrics compete (e.g. precision and recall), combine them into one (e.g. F1) rather than carrying both, so there is always an unambiguous “better.”

2.2 Optimizing vs. satisficing metrics

When you care about several things that don't reduce to one number, split them: pick **one optimizing metric** to maximize, and treat the rest as **satisficing constraints** that merely need to clear a threshold. What lands in which bucket is determined by **stakeholder priority** — the same quantity can be optimizing in one project and satisficing in another.

Metric type	Goal	How many	Example (sign detector)
Optimizing	Do as well as possible	Exactly one	Accuracy — maximize it
Satisficing	Just clear a threshold	Zero or more	Runtime ≤ 10 s; memory ≤ 10 MB; accuracy $\geq 98\%$

Worked rule Among models, first **disqualify any that violate a satisficing constraint**, then pick the highest optimizing score from those that remain. The highest-accuracy model is the wrong choice if it blows the runtime budget.

2.3 When to change the metric or the dev/test set

If the current metric ranks a model you consider worse above one you consider better — or if real-world performance diverges from dev-set performance — the metric or the data no longer captures what you care about.

- **Higher false-negative cost than expected?** Reset the team's target (re-weight the metric so the costly error is penalized), then tune to it. Don't optimize a single raw rate that can be gamed (driving false negatives to zero by flagging everything).
- **Distribution shifted** (e.g. a new bird species migrates in)? **Redefine the dev/test set first** so it reflects the data you now care about — only then resume improving the model.

3. Setting Up Train / Dev / Test Sets

3.1 Dev and test must come from the same distribution

The dev and test sets together **define the target**. They must be drawn from the same distribution, and that distribution must reflect the data you most care about performing well on. Tuning against a dev set that doesn't match test (or doesn't match deployment) means “aiming where you don't want to hit.”

Common pitfall Adding off-distribution images only to the dev set splits dev from test **and** pulls dev away from the data you care about. Off-distribution data belongs in the **training** set, where extra volume helps and distribution mismatch is tolerable.

3.2 Size dev/test in absolute terms, not by ratio

The old 60/20/20 and 70/30 splits assumed small datasets. With millions of examples, dev and test only need to be **large enough to evaluate confidently** — everything else should go to training.

Dataset size	Sensible split	Rationale
~1,000–10,000	60 / 20 / 20	Need a large fraction to get a stable estimate
10,000,000	~95 / 2.5 / 2.5 (e.g. 9.5M / 250k / 250k)	250k is ample to evaluate; rest maximizes training

3.3 Put the right data in the right place

- **Extra off-distribution data** (e.g. internet images) → add to the **training set**. It can improve learned features even if it isn't a perfect match.
- **Never** reshuffle off-distribution data into dev/test, and never enlarge dev/test “just because” — a 3M test set wastes data the model could have trained on.

4. Human-Level Performance, Bias & Variance

4.1 Why human-level performance matters

Human-level performance is used as a **proxy for Bayes error** — the lowest error any function could achieve. To make the proxy as tight as possible, define it using the **best** available human (lowest error), e.g. the best expert at 0.3%, not an average that includes non-experts. A loose baseline makes a model look like it has no room to improve when it actually does.

4.2 The bias / variance decomposition

Two gaps drive every “what next” decision:

- **Avoidable bias** = training error – human-level (Bayes proxy). Large → attack bias.
- **Variance** = dev error – training error. Large → attack variance.

Problem	Signal	Effective knobs	Avoid
High avoidable bias	Big train–human gap	Bigger / more complex model; train longer; better architecture; decrease regularization	Adding data; more regularization
High variance	Big dev–train gap	More training data; increase regularization	Just adding model capacity

Watch out Bias is only meaningful **relative to a baseline**. Given training error = 4.0% and dev = 4.5% but **no** human-level figure, you **cannot** tell whether 4.0% is high bias — there is insufficient information. The 0.5% gap tells you variance is low; bias is unknowable without Bayes context.

4.3 Bayes error and surpassing humans

- An algorithm **can** exceed human-level performance (humans aren't always optimal) but **can never beat Bayes error** — the absolute floor set by irreducible noise.
- An **achieved** error rate is an upper bound on Bayes error: if a model reaches 0.05%, then Bayes error $\leq 0.05\%$.
- Once you **surpass human-level**, avoidable bias becomes hard to measure (you've lost the Bayes proxy), so progress typically slows.
- Never assume Bayes error is 0% — the remaining gap may be irreducible, not “quick to close.”

5. Bias & Variance with Mismatched Distributions

When training data comes from a different distribution than dev/test, a single train–dev gap conflates two different problems. Carve out a **training-dev set** — held out from training, same distribution as training — to separate them.

Gap measured between	Diagnoses	If large, do this
Human-level → Training	Avoidable bias	Bigger model, train longer
Training → Training-dev	Variance	More data, regularization
Training-dev → Dev	Data mismatch	Make training data more like dev; synthesis
Dev → Test	Overfitting to dev	Get a bigger dev set

Worked example. Train 1% / Train-dev 5.1% / Dev 5.6%, human 0.5%. Avoidable bias 0.5%, variance **4.1%** (dominant), data mismatch 0.5% → a **high-variance** problem, not data mismatch.

Addressing data mismatch has no fully systematic fix. Do manual error analysis on the **dev set** (not test) to characterize how it differs from training, then make training data more similar — often via artificial data synthesis (e.g. adding car noise, synthesizing fog).

Synthesis caveat Synthesizing from a small pool (e.g. 1,000 fog images) risks the model **overfitting to that narrow subset**, even though every result looks realistic to a human. “Looks realistic to people” does not guarantee the synthetic data covers the true distribution.

6. Workflow & Error Analysis

6.1 Build your first system quickly, then iterate

ML is highly empirical. The fastest way to learn what matters is to build a quick-and-dirty baseline and let **error analysis** point to the real problems — rather than spending days speculating about difficulties or collecting data before you know it's the bottleneck.

6.2 Error analysis

- Manually examine the **misclassified dev-set** examples (e.g. 500 of them), one at a time, and tally the causes.
- Never run error analysis on the **test set** — that would tune you to it and compromise its role as an unbiased estimate.
- Use the tallies to **size each error category** and allocate effort. The largest single category isn't automatically where all effort should go: if occlusion is 7.2% but the other causes sum to 8.1%, a fix targeting only occlusion has a lower ceiling than addressing the rest.

6.3 Cleaning up incorrectly labeled data

- Deep learning is fairly robust to **random** label errors in the **training** set — fixing them is optional and often impractical.
- If you correct labels in the dev set, you **must apply the identical correction to the test set** so they stay from the same distribution.
- Also examine examples the model got **right**, not only the ones it got wrong — otherwise you bias the correction.
- Knowing a category contributes X% of errors tells you **whether the fix is worth it**; it does **not** mean removing it subtracts exactly X% from overall error.

7. Choosing a Learning Paradigm

7.1 Transfer learning

Pre-train on a large source task, then **fine-tune** on the small target task. Most valuable when the source has **far more data** than the target and they share low-level features (same input type).

- Example: a large red/green light model fine-tuned on a tiny yellow-light dataset. Don't subsample the source to “balance” it — that discards the very features transfer relies on.

7.2 Multi-task learning

One network predicts **several labels at once** (e.g. stop sign AND red light in the same image), sharing features across tasks.

- The loss **sums per-task losses**, so **partially-labeled** images are usable: simply omit the unknown entries (the “?”) from the sum.
- Multi-label → use a **sigmoid** output per class (independent yes/no), **not** softmax (which forces mutually-exclusive classes). Several **mutually-exclusive** categories → softmax instead, which is single-label classification, not multi-task.

7.3 End-to-end deep learning

A **single network maps raw input straight to the final output**, replacing a hand-engineered pipeline. “Image → light present?” in one net is end-to-end; “detect the light, then classify its color” is a pipeline.

- End-to-end often needs **a lot of data**; pipelines can win when data is limited or when sub-tasks have abundant data of their own. “More end-to-end” is not automatically “better.”

Paradigm	Use when	Output / key detail
Transfer learning	Large source task, small target task, shared inputs	Pre-train → fine-tune later layers
Multi-task learning	Many labels co-occur per image, comparable data per task	Shared net; sigmoid outputs; tolerate missing labels
End-to-end	Plentiful input→output data; want to drop hand-engineering	Single net, raw input to final answer

8. One-Page Diagnostic Cheat Sheet

Symptom	Diagnosis	First move
Big training–human gap	Avoidable bias	Bigger model; train longer; less regularization
Big dev–training gap (same distribution)	Variance	More data; more regularization
Big training-dev → dev gap	Data mismatch	Error-analyze dev; make train resemble dev; synthesize
Big dev → test gap	Overfit to dev	Get a bigger dev set
Metric ranks a worse model higher	Wrong metric	Redefine/re-weight the metric, then tune to it
Real-world worse than dev	Wrong dev/test distribution	Rebuild dev/test from data you care about
Several competing requirements	Need metric structure	One optimizing metric + satisficing thresholds
Model beats human-level	Near/at Bayes error	Expect slower progress; bias hard to measure
Small, rare target dataset	Data-starved task	Transfer learning (pre-train → fine-tune)
Multiple objects per image	Multi-label	Multi-task net with sigmoid outputs

The throughline Separate **aiming** (metric + dev/test target) from **hitting** (bias, variance, mismatch). Fix the target first; then turn one orthogonal knob at a time, guided by error analysis rather than guesswork.